

AIで防災アプリを作ってみて

開発を通して学んだこと

開発メンバー勉強会 / 課題1

Kyoko Takazawa

目次

1. 課題と要件
2. 作ったもの
3. 開発方法
4. 学んだこと
5. まとめ

課題と要件

- 課題1：アプリケーションを一つ、自分で作る
- 必須要件：**HTTPS** / **OIDC認証** / **Git管理**
- テーマは自由。何を作るかも自分で決める

作ったもの：災害時避難シミュレーター

- 災害時に、避難所と避難ルートを提案する Web アプリ
- 災害種別（地震・津波・台風・洪水・土砂）で提案が変わる
- 危険な浸水区域を避けるルート、オフライン対応
- 公開URL : <https://kyokoron.github.io/team-challenge/>

開発方法

- AI（Claude Code）と対話しながら開発
- コード作成は AI、要件定義・レビュー・判断は自分が担当
- 構成：GitHub の静的サイト（サーバー・DB なし）、認証は Auth0（OIDC）
- 開発から公開まで、すべてブラウザで完結

学んだこと① AIの出力は検証が必要

- AI は、もっともらしい回答を即座に出す
- 例：「DB が必要なのは投稿・更新のときだけ」と断定
→ 問い直すと、オフライン対応・更新頻度・コストなど論点は多かった
- 出力を鵜呑みにせず、**検証と問い直し**が欠かせない

学んだこと②「動く」と「正しい」は別

動作はするが、誤った挙動が複数あった。

- 内陸で「津波」を選ぶと、海側へ誘導していた（避難所が沿岸に偏るため）
- データ取得に失敗すると、実在しない避難所を表示していた
- 広域データで距離計算が狂い、最寄り順が崩れていた
- 直線距離を、そのまま所要時間として表示していた

危険時は垂直避難を促す、不正なデータは表示しない、など安全側へ修正した。

学んだこと③ データ整備に手間がかかる

- 避難所・ハザード情報は、国のオープンデータで揃う（無料）
- ただし実際に使うには課題があった
 - どれが正解のデータか分かりにくい
 - 洪水浸水想定は河川単位で分割されている
 - 古い形式（Shapefile）で、変換・整形が必要

使える形に整える前処理に、実装と同程度の時間がかかった。

学んだこと④ AI時代、価値の重心が移る

これまで

ユーザー

↓ UIを操作

UI (アプリ)

↓

機能 (ロジック)

↓

データ

価値の中心 : UI設計・機能開発・実装

これから

ユーザー

↓ 目的を伝える

AI (理解・推論)

↓

データ

価値の中心 : ドメイン理解・データ整備・課題設定

実装の速さは AI に任せられる。人が担うのは、ドメイン理解・データ整備・課題設定。

まとめ

- AI により、開発の速度は大きく向上する
- 一方で、正しさ・安全性の判断は人が担う必要がある
- 作ること以上に、「何を・正しく作るか」を見極めることが重要だと感じた

付録：技術・データ

主な機能

- 災害種別に応じた避難所ランキング
- 浸水域を避ける避難ルート
- 現在地が危険なときの警告
- オフライン（PWA）／OIDC認証 + HTTPS

技術・データ

- Vanilla JS / MapLibre GL JS / Auth0(OIDC)
- Service Worker + IndexedDB
- 国土地理院（地図・避難所・標高）
- 国土数値情報（洪水浸水想定 A31）
- OpenRouteService（回避ルート）